



SOFTWARE ENGINEERING FOR SCIENTISTS

Best Practices

Ten habits that make research code readable, reliable, and reusable.

A guide for writing code other people — including future you — can trust.



OVERVIEW

Ten Habits, One Repository



01

Manage Dependencies with Conda / Mamba

Isolate environments; pin dependencies



02

Follow a Style Guide

PEP8 formatting, checked automatically



03

Document and Comment Your Code

Comments for developers, docstrings for users



04

Use Small Functions That Do One Thing

Small, single-purpose, side-effect-free



05

Use the main() Function

Guard top-level code with `__name__`



06

Use argparse for Input Parameters

Named, order-free command-line arguments



07

Handle Your Exceptions

Catch failures; exit with the right code



08

The Git Branch & Pull Request Workflow

Branch, push, pull request, merge, tag



09

Organize Your Repository

src, test, doc, .gitignore, LICENSE



10

Write a Good GitHub README

The first thing every user reads



01 • DEPENDENCIES

Manage Dependencies with Conda / Mamba

Dependencies pile up fast, and getting a version even slightly wrong can silently break a library. Use mamba — a fast, drop-in replacement for conda — to resolve and install exactly what each project needs, in its own isolated environment.

install Mambaforge — once per machine

```
$ cd $HOME
$ curl -L -O https://github.com/conda-forge/
  miniforge/releases/latest/download/
  Mambaforge-pypy3-Linux-x86_64.sh
$ bash Mambaforge-pypy3-Linux-x86_64.sh
```

create & activate — per project

```
$ mamba create --yes -n swe4s
$ conda activate swe4s
(swe4s) $
```



Why mamba, not conda? Conda became slow as it grew; mamba is a transparent, faster replacement for the conda command. Anywhere you see \$ conda, you can safely use \$ mamba instead.



01 • DEPENDENCIES

Install Packages, Then Save the Recipe

Install into the active environment with mamba; fall back to pip only for packages conda doesn't have. Then capture everything in `environment.yml` so collaborators — and your CI — can recreate it exactly.

```
with conda

(swe4s) $ mamba install -y \
    pycodestyle
```

```
with pip

(swe4s) $ pip install \
    pycodestyle
```

environment.yml — hand-curated

```
name: swe4s
channels:
  - conda-forge
dependencies:
  - python
  - pycodestyle
```



Start from `mamba env export --from-history`.



Keep only the packages your project truly needs.



Remove the prefix line — it names your machine.

Not portable: the plain `mamba env export` output depends on your OS and whatever else is installed — an environment built on a Mac is unlikely to work on Linux.



02 · CODE STYLE

Follow a Style Guide

Code is read far more often than it is written. PEP8 removes the ambiguity about what counts as well-formatted Python — comments matter, but they are never a substitute for clean, consistent code.



AVOID

```
foo = long_function_name(var_one, var_two,  
    var_three, var_four)
```



PREFER

```
foo = long_function_name(var_one, var_two,  
    var_three, var_four)
```



The standard: PEP 8 (python.org/dev/peps/pep-0008) is the comprehensive style guide for this course. Your code should follow it unless you have a good reason not to — it is broken into sections with many worked examples.



Check It Automatically With pycodestyle

Install with pip or conda, then lint any file in one command. Add --show-source and/or --show-pep8 for more detail.

```
style_test.py
import sys, os

foo = long_function_name(var_one, var_two,
                          var_three, var_four)

a=10
if(a<20): print('small')
```

```
$ pycodestyle style_test.py

$ pycodestyle style_test.py
style_test.py:1:11: E401 multiple imports
style_test.py:3:43: W291 trailing whitespace
style_test.py:4:5: E128 under-indented
style_test.py:6:2: E225 missing whitespace
style_test.py:7:5: E225 missing whitespace
style_test.py:7:9: E701 multiple statements
style_test.py:8:1: W391 blank line at EOF
```

Why it matters: a linter catches formatting drift before review, so human feedback can focus on logic and design instead of spacing and imports.



03 · DOCUMENTATION

Document and Comment Your Code

Open-source adoption alone will not make your project reproducible. Comments and documentation serve different readers — know which one you are writing for.



Comments

Short, plain notes for developers (you and your lab). Start with #, stay brief, and sit as close to the code as possible.



Documentation

Written for your users — docstrings that explain functionality so they can apply your code to their own research.

Comments in context

```
# The list is often sorted, so we don't use quick sort
V_sorted = heapsort(V)

# TODO: sort this list then do binary search
for i in range(len(V)):
    if V[i] == key:
        hit = i

# BUG: catch the execution for an empty list
mean = sum(V)/len(V)
```



03 · DOCUMENTATION

Documenting With Docstrings

Docstrings are strategically placed strings wrapped in `"""triple quotes"""` that populate an object's `__doc__` field. We use the numpydoc format — focus on the details that matter most and skip the rest.

math_functions.py

```
def get_array_mean(V):  
    """Compute the arithmetic mean.  
  
    Parameters  
    -----  
    V : list of int  
        Non-empty array of numbers.  
  
    Returns  
    -----  
    m  
        Arithmetic mean of V  
    """  
    m = sum(V)/len(V)  
    return m
```

```
>>> help(get_array_mean)  
  
>>> help(math_functions.get_array_mean)  
  
get_array_mean(V)  
    Compute the arithmetic mean.  
  
    Parameters  
    -----  
    V : list of int  
        Non-empty array of numbers.  
  
    Returns  
    -----  
    m  
        Arithmetic mean of V
```



04 • FUNCTIONS

Use Small Functions That Do One Thing

"The first rule of functions is that they should be small. The second rule is that they should be smaller than that." — Clean Code, Robert C. Martin



Efficiency

Write and test once, reuse everywhere — no rewriting equivalent code.



Maintainability

Fix a bug in one place and every caller benefits immediately.



Reduced Error

The more a piece of code is reused, the faster its bugs surface.



Easier Testing

Small components can be tested in isolation, reliably.



Readability

Modular code is clearer — functionality is understood in isolation.

Six rules for writing them:

- 1 Small — 10 lines is a good target; higher-level functions may run longer.
- 2 Do one thing — the most important rule; multiple jobs hurt reuse and testing.
- 3 Use descriptive names — a name you can't write clearly is a function that breaks rule 1 or 2.
- 4 Prefer fewer arguments — group related inputs instead of a long parameter list.
- 5 Have no side effects — return values instead of mutating something outside the signature.
- 6 Avoid flag arguments — split into separate methods instead of branching on a boolean.



04 • FUNCTIONS

Side Effects Break Rule 2

Side effects rely on — or modify — something outside the function signature. They reduce reusability and, worse, invite bugs when the effect is inevitably forgotten. If you **MUST** have one, document it.



WITH SIDE EFFECT

```
a = [ , , , ]
def get_mean_with_sideeffect(a):
    total = 0
    for i in range(len(a)):
        total += a[i]
    a.append(total/len(a))

get_mean_with_sideeffect(a)
```



WITHOUT SIDE EFFECT

```
a = [ , , , ]
def get_mean_without_sideeffect(a):
    total = 0
    for i in range(len(a)):
        total += a[i]
    return total/len(a)

mean = get_mean_without_sideeffect(a)
```



05 • ENTRY POINTS

Use the `main()` Function

Python executes every line of a source file, whether it is run directly or imported as a module. Import a file without a guard, and its top-level code runs anyway.

```
math_lib.py — no guard
def list_sum(L): ...
def list_product(L): ...

V = [1, 2, 3, 4, 5, 6]
print(list_sum(V))
print(list_product(V))

# foo.py
import math_lib
X = [10, 20, 30, 40, 50]
print(math_lib.list_sum(X))
```

```
$ python foo.py

$ python foo.py
21
720
150
12000000

# 21 and 720 leaked in
# from math_lib on import
```



The problem: unguarded top-level statements execute unintentionally the moment the file is imported — not just when it is run directly.



05 • ENTRY POINTS

The Fix: Guard With `__name__`

Wrap the code you only want to run directly inside a `main()` function, then guard the call — code that is imported as a library stays quiet.

math_lib.py — guarded

```
def list_sum(L): ...
def list_product(L): ...

def main():
    V = [1, 2, 3, 4, 5, 6]
    print(list_sum(V))
    print(list_product(V))

if __name__ == '__main__':
    main()
```

 \$ python foo.py

```
$ python foo.py
150
12000000
```



`__name__ == '__main__'`

is only True when the file is executed directly — never on import.



Use argparse for Input Parameters

Don't rely on the user to remember parameters and their order. Reading `sys.argv` by position gives cryptic errors the moment something is missing or out of place.

⊗ THE BAD WAY

```
the_bad_way.py
import sys

file_name = sys.argv[1]
column_number = int(sys.argv[2])
```

Forget a parameter, give the wrong count, or pass them in the wrong order — every case is an unhelpful traceback, not a clear message.

```
three ways it breaks

$ python the_bad_way.py
IndexError: list index out of range

$ python the_bad_way.py test.txt
IndexError: list index out of range

$ python the_bad_way.py 1 test.txt
ValueError: invalid literal for int()
with base 10: 'test.txt'
```



The Good Way: argparse

```
the_good_way.py
import argparse

parser = argparse.ArgumentParser(
    description='The right way.')

parser.add_argument('--file_name',
    type=str, required=True)

parser.add_argument('--column_number',
    type=str, required=True)

args = parser.parse_args()
```



Order does not matter — every value is named.



Missing or bad arguments get a clear, formatted error.



The -h flag prints full usage automatically.



Handle Your Exceptions

Unhandled runtime errors leave users staring at a technical traceback. Wrap risky code in a try/except handler — generic to catch anything, or specific per exception type.

GENERIC

```
A = [1, 2, 3, 4]

try:
    print(A[4])
except:
    print('Something went wrong')
```

SPECIFIC — BY EXCEPTION TYPE

```
try:
    f = open(file_name, 'r')
except FileNotFoundError:
    print('Could not find ' + file_name)
except PermissionError:
    print('Could not open ' + file_name)
finally:
    return f
```

Built-in exceptions cover most cases (docs.python.org/3/library/exceptions.html); extend Exception for your own. A single function can raise different exception types depending on what went wrong — catch each one where it means something different to the user.



07 · ERROR HANDLING

Set Exit Codes for Every Failure

When a program exits, it returns an integer to the operating system. 0 means success; 1-255 means error. If your script exits on an unhandled error mode, set the code explicitly with `sys.exit()`.

0

Success

Program finished with no errors. This is the default if the script never exits abnormally.

1-255

Error

1 is the catch-all for general errors; an unhandled exception sets this to 1 automatically.

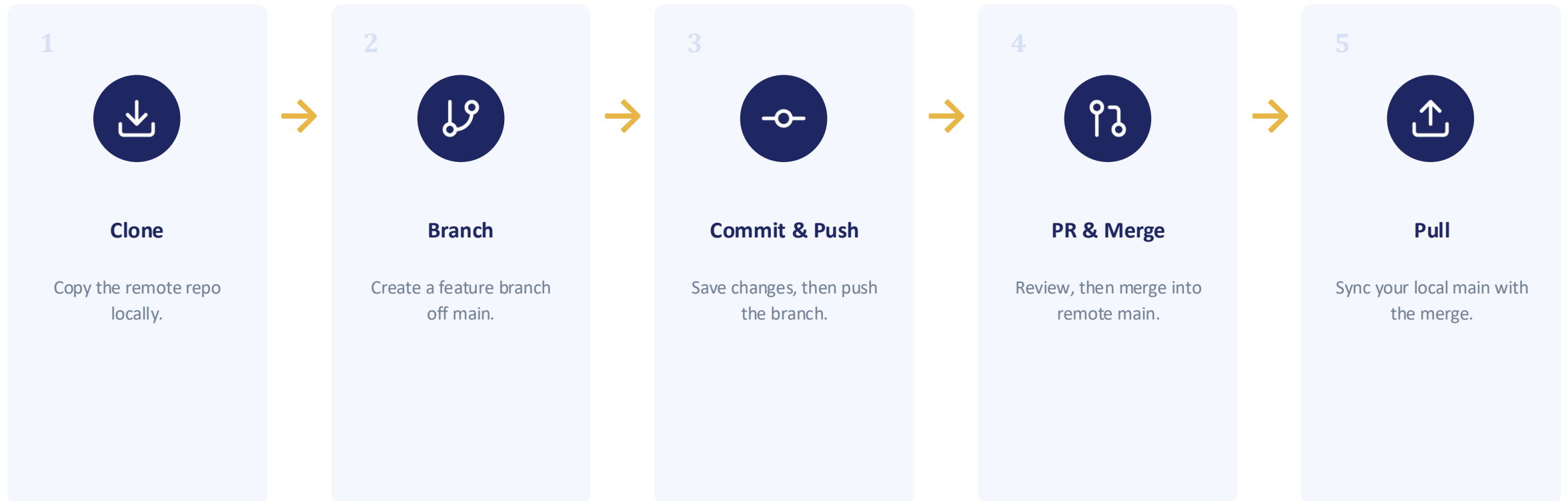
```
exit_code.py

$ python exit_code.py 1 0      # sys.exit(1) on divide-by-zero
$ echo $?
1
$ python exit_code.py 1 1      # unhandled exception sets code to 1
1
$ echo $?
0
```



The Git Branch & Pull Request Workflow

To get the most out of version control, never commit directly to your local or remote main branch. Branch, push, open a pull request and merge, then pull the result back to your local main.





08 · VERSION CONTROL

Tag the Points Worth Remembering

Tags capture a point in your repository's history. Follow semantic versioning — major, minor, patch (e.g. v1.2.3) — so collaborators know at a glance what changed.



Version Milestones

Mark development milestones you plan to distribute.



Stable Releases

Make it easy for collaborators to find the stable version.



Hotfixes

Give easy access to bug fixes in a prior release.



Beta / Alpha

Share early versions with testers, e.g. v1.2.3-beta.



`git tag -a v1.2.3 -m "message"` creates an annotated tag — push it explicitly with `git push --tags`.



09 · PROJECT LAYOUT

Organize Your Repository

Your repository is the public face of your project. A well-structured layout makes it easier for collaborators to contribute, for users to understand your work, and for you to maintain it after a long break.



src/

All project source code, not including tests. Larger projects will nest sub-directories here.



test/

All test code — split into unit/ and func/ once you have enough of each to warrant it.



doc/

Scientific documentation or technical specifications for the project.



.gitignore

Optional but recommended — keeps OS and test-run temp files out of the repo.



LICENSE

Optional, but without one your public code isn't truly open source (MIT, GPL, Apache 2.0...).



README.md

The first thing people see — rendered from Markdown, it explains purpose and usage.



Write a Good GitHub README

A README.md is automatically rendered by GitHub whenever someone opens a folder in your repository — the one in the root is shown on the main page. No README means nothing is displayed at all.

README.md (source)

```
# Less is more
## More is less
```

```
*somewhat important* or
**very important**
```

```
Links [Pages] (http://github.com/)
```

```
- unordered
- list
```

```
1. ordered
2. list
```

renders as

Less is more

More is less

somewhat important or **very important**

Links [Pages](#)

- | | |
|-------------|------------|
| • unordered | 1. ordered |
| • list | 2. list |



What to Include, At a Minimum



A Short Description

Your elevator pitch in text form. Mention the scientific field, statistical models, and input data — more specific than an abstract in some places, more general in others.



How to Use It, With Examples

Go beyond the usage argparse prints — cover the format of input files and how different parameter combinations change the resulting behavior.



How to Install It

Never assume a user's system configuration. Give a step-by-step guide with specific commands to install dependencies and run tests — conda is a useful common starting point.



KEY TAKEAWAYS

Ten Habits, One Sentence Each

01

Conda/Mamba — isolate environments per project and pin dependencies you can reproduce.

02

Style — let PEP8 and pycodestyle settle formatting so review time goes to logic.

03

Comments & docs — write for developers and for users, separately.

04

Functions — small, single-purpose, and free of side effects.

05

main() — guard top-level code so imports stay quiet.

06

argparse — named, order-free, self-documenting command-line input.

07

Exceptions — catch failures deliberately and exit with the right code.

08

Git — branch, push, pull request, merge, then pull; tag what matters.

09

Repository layout — src, test, doc, .gitignore, LICENSE, README.

10

README — the first thing every user reads; make it count.